MLOps Face Recognition System

Full Pipeline Project Report

Name & Roll:	Rajshekhar Rakshit — CS24S031
System:	Siamese Face Recognition — Full MLOps Pipeline
Components:	Flask API · Airflow DAG · Trainer · MLflow
	Prometheus/Grafana · Docker Compose
Technology:	PyTorch · Optuna · Airflow 3.1.8 · MLflow 2.11.1
Document:	Design, Implementation & MLOps Review
Date:	April 2026

This report covers the complete AI product lifecycle for a self-improving, privacy-preserving face recognition service built entirely on commodity hardware with no cloud dependencies, following the MLOps guidelines document.

Contents

1 Introduction and Problem Definition

1.1 Business Understanding

This project delivers a **self-improving, privacy-preserving face recognition service** deployable on commodity hardware with no cloud dependencies. The system addresses the business need for real-time identity verification in environments where biometric data must remain on-premises for compliance and privacy reasons.

1.1.1 Measurable Success Metrics

ML Metrics:

- Triplet loss on validation set < 0.3
- Nearest-neighbour batch accuracy $> 85\%$ at inference threshold 0.8

Business / Operational Metrics:

- Inference latency < 200 ms per frame (Prometheus histogram)
- Model hot-reload without downtime (zero-downtime upgrade)
- Re-training triggered within minutes of misclassification threshold breach
- All biometric data remains within the deployment environment

1.2 System Scope

The scope spans six Docker Compose subsystems: inference serving, model training, experiment tracking, workflow scheduling, observability, and persistent storage. The system is designed to run entirely on local or on-premises hardware, explicitly forbidding cloud platforms per the project constraints.

2 Data Collection and Validation

2.1 Dataset: LFW (Labeled Faces in the Wild)

The primary training dataset is the **Labeled Faces in the Wild (LFW)** benchmark. Only identities with a minimum of 2 images are retained (`min_images_per_identity = 2` in `params.yaml`). The dataset is split at the *identity level* (not image level) to prevent data leakage:

Split	Ratio	Purpose
Train	70%	Triplet loss optimisation
Validation	15%	Hyperparameter selection, early stopping
Test	15%	Pair-based final evaluation

Table 1: Identity-level dataset split (configured in `params.yaml`)

2.2 Data Validation (`prepare_data.py`)

The `prepare_data.py` stage performs four automated validation checks before any training can proceed:

1. **Directory existence check** — raises `FileNotFoundError` with actionable guidance if the dataset is absent.
2. **Schema / structure check** — `scan_lfw()` enforces the expected `<identity>/<image>.jpg` hierarchy.
3. **Minimum-image constraint** — identities with fewer than `MIN_IMAGES` are excluded.
4. **Corruption sampling** — 5% of all images are opened with `PIL.Image.verify()` to detect corrupt files; warnings are logged.

2.3 Misclassification Feedback Loop — Data Augmentation

A key differentiator of this system is the **Human-in-the-Loop (HITL)** data pipeline. When the Flask API captures a misclassification report, the face crop is saved to `data/misclassified/<person>/`. Before each training run, `merge_misclassified_into_train()` merges these crops into the training split *only*, ensuring that:

- Corrected crops never leak into validation or test sets.
- Each registered identity's real-world appearance data feeds future fine-tuning.

2.4 Data Security

Per the guidelines requirement that all sensitive data must be encrypted at rest and in transit, the system stores face embeddings and misclassification records in an SQLite database. Biometric face crops are stored exclusively within the Docker volume scope; no data is transmitted to external servers during inference.

3 Data Preprocessing and Feature Engineering

3.1 Image Preprocessing Pipeline

Preprocessing is split into training-mode (with augmentation) and evaluation-mode (deterministic) transforms, implemented in `dataloader.py`:

Step	Description	Split
Resize(256)	Rescale shorter edge to 256 px	Both
RandomCrop(224) / CenterCrop(224)	Final crop to model input size	Train / Val&Test
RandomHorizontalFlip(p=0.5)	Horizontal mirror augmentation	Train only
ColorJitter	Brightness, contrast, saturation, hue perturbation	Train only
RandomRotation(±10°)	Rotational robustness	Train only
ToTensor()	Convert PIL image to float tensor	Both
Normalize(ImageNet)	Subtract ImageNet mean, divide by std	Both

Table 2: Image transformation pipeline (`get_transform()` in `dataloader.py`)

3.2 Baseline Statistics for Drift Detection

As required by the guidelines, per-split pixel statistics are computed and persisted during `prepare_data.py`:

- **Per-split pixel mean and standard deviation** (RGB channels) — computed over a 500-image random sample per split.
- **Pixel variance across samples** — used as a baseline for incoming data distribution comparison.
- Written to `data/{dataset}_baseline_stats.json` for later drift comparison.

3.3 PKSampler — Structured Batch Construction

Standard random sampling is replaced by **P×K sampling** (`PKSampler` in `dataloader.py`). Each batch contains exactly P identities with K images each, guaranteeing a minimum ratio of positive pairs per batch — essential for stable triplet loss training. The relationship is:

$$\text{batch\_size} = P \times K \qquad (P = \text{identities},\ K = \text{images per identity})$$

3.4 Feature Store Concept

Per the guideline requirement, feature engineering logic is versioned separately from model logic. All transform definitions reside in `dataloader.py` (a distinct module from `model.py` and `train.py`), and augmentation parameters are externalised to `config.py` / `params.yaml` for reproducible versioning.

4 Model Development and Training

4.1 Model Architecture

4.1.1 Siamese Network with ResNet50 Backbone

The model is a **Siamese Network** with a shared-weight encoder, defined in `model.py`. The design choice of a Siamese architecture is motivated by the fundamental nature of the

task: face verification is *metric learning* — determining whether two images belong to the same identity.

Component	Details
Backbone	ResNet50 (ImageNet V2 pre-trained, frozen during Phase 1)
Projection head	Linear(2048→512) → BN → ReLU → Dropout(0.3) → Linear(512→128)
Output	L2-normalised 128-dimensional embedding
Similarity metric	Cosine distance (L2 pairwise distance on normalised vectors)
Loss function	Triplet loss (configurable: `all` / `hard` / `semi-hard` mining)

Table 3: SiameseEncoder architecture summary

4.1.2 Why ResNet50?

Three factors drove the backbone selection:

1. **Bias–variance trade-off**: Deep enough to capture discriminative facial geometry, yet not over-parameterised for small per-identity sample counts.
2. **Transfer learning**: The pre-trained checkpoint from `torchvision` dramatically reduces the labelled data requirement.
3. **Training speed**: Freezing the backbone during warm-up (Phase 1) keeps the weekly re-training cadence feasible on commodity hardware.

4.1.3 Resource Constraints and Optimisation

Per the guidelines requirement to optimise for local/on-prem hardware:

- Backbone is **frozen** during the warm-up phase, reducing trainable parameters from 25.6M to ∼0.7M.
- `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` is set in `docker-compose.yaml` to reduce GPU memory fragmentation.
- Gradient clipping (`max_norm=5.0`) prevents training instability on small batch sizes.
- `torch.cuda.empty_cache()` is called after each epoch and trial to reclaim GPU memory.

4.2 Two-Phase Training Strategy

Training is split into two phases managed by `train.py`:

Phase	Epochs	Backbone	Learning Rate
1 (Warm-up)	1 → `WARMUP_EPOCHS`	Frozen	`LEARNING_RATE` (cosine decay)
2 (Fine-tune)	`WARMUP_EPOCHS+1` → `NUM_EPOCHS`	Unfrozen	`BACKBONE_LR=1e-5`, `FINETUNE_LR=1e`

Table 4: Two-phase training schedule

4.3 Triplet Loss with Hard Mining

The `TripletLoss` class in `utils.py` supports three mining strategies:

- **All**: Every valid (anchor, positive, negative) triplet contributes to the loss.
- **Hard**: Hardest positive (maximum distance) and hardest negative (minimum distance) per anchor.
- **Semi-hard** (default): Negatives that are farther than the positive but within the margin — the most stable strategy for early training.

4.4 Experiment Tracking with MLflow

Every training run logs to MLflow (`face-recognition-siamese` experiment):

- **Parameters**: `batch_size`, `learning_rate`, `margin`, `triplet_mining`, `warmup_epochs`, `embedding_dim`, `threshold`.
- **Metrics per epoch**: `train_loss`, `train_acc`, `val_loss`, `val_acc`, `learning_rate`.
- **Artefacts**: Best model checkpoint (`best_model.pth`), training history JSON.
- **Tags**: `sweep_id`, `trial_index`, `best_of_sweep` for sweep-level filtering.

4.5 Hyperparameter Optimisation with Optuna

`sweep_optuna.py` implements Bayesian hyperparameter search using Optuna's TPE sampler. The search space is defined in `params.yaml`:

Hyperparameter	Range	Sampler
Learning rate	[`lr_min`, `lr_max`]	Log-uniform
Triplet margin	[`margin_min`, `margin_max`]	Uniform
Mining strategy	`mining_choices` (e.g., `semi,hard`)	Categorical

Table 5: Optuna sweep search space from `params.yaml`

Each trial is a full training run tagged with a shared `sweep_id` in MLflow, enabling side-by-side comparison. The best trial by validation loss is written to `checkpoints/{dataset}_best_run.json` for the downstream registration stage.

Adaptive sweep size: When `TRIGGERED_BY=misclassification_threshold`, the number of trials is automatically reduced to 1 (fast fine-tuning rather than full exploration), balancing computational cost against responsiveness.

4.6 Reproducibility

Per the guidelines requirement that every experiment must be reproducible via a Git commit hash and MLflow run ID:

- All hyperparameters are externalised to `params.yaml` (committed to Git).
- DVC tracks the dataset and artefact lineage (`MLproject` file).

- Every run produces a unique `run_id` stored in `logs/mlflow_run_id.txt`.
- The `params.yaml` + `run_id` pair is sufficient to fully reproduce any historical training run.

5 Model Deployment

5.1 Deployment Strategy: Online Serving

The system uses an **online inference** strategy with real-time video stream processing. The Flask API (`app.py`) serves inference at `localhost:8080` (mapped from container port 2000).

5.2 Multi-Container Strategy

The deployment follows the guideline requirement of a multi-container Docker Compose setup:

Container	Port	Role
`flask-api`	8080:2000	Inference API, face registration, metrics
`mlflow`	5000	Model registry, experiment tracking
`trainer`	—	Training container (launched by Docker-Operator)
`airflow-webserver`	8081:8080	Pipeline orchestration UI
`airflow-scheduler`	—	DAG scheduling daemon
`prometheus`	9090	Metrics scraping
`grafana`	3001:3000	Monitoring dashboards
`postgres`	—	Airflow metadata database

Table 6: Docker Compose service topology

5.3 Health Checks

Per the guideline requirement for `/health` and `/ready` endpoints:

- The `flask-api` container defines a Docker healthcheck:
  `python -c "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http:/`
  `else 1)"`
- The Airflow webserver healthcheck polls `/api/v2/monitor/health` every 10 seconds.
- Downstream services (`airflow-init`, `airflow-scheduler`) depend on `condition: service_healthy`.

5.4 Zero-Downtime Model Upgrade

The system achieves zero-downtime model upgrades through a three-step protocol:

1. Training pipeline promotes the new model to **Production** in MLflow registry.

2. Airflow DAG calls `/notify_flask` (or the Flask API polls `MODEL_RELOAD_INTERVAL_SECS=60`).
3. The Flask API acquires a thread lock (`_model_lock`), hot-reloads the new model, and performs a full re-embedding pass over all registered faces to correct embedding shift.

5.5 Remote GPU Support

When `RUN_REMOTE=true`, the Airflow DAG SSHes into the configured remote GPU host and dispatches the training job there, allowing lightweight inference hardware to coexist with heavy training workloads.

5.6 Model Versioning and Rollback

The MLflow Model Registry maintains two stages:

- **Staging**: Post-training candidate registered by `register_model.py`.
- **Production**: Currently active model. The `evaluate_and_promote` DAG task only advances a model to Production if its validation metric improves on the current Production baseline — providing an automatic rollback safety net.

6 Model Monitoring and Maintenance

6.1 Performance Monitoring with Prometheus and Grafana

Prometheus scrapes the Flask API's `/metrics` endpoint every 15 seconds. The following metrics are tracked:

Metric	Type
`face_misclassification_count`	Gauge — pending HITL errors
Recognition latency	Histogram — inference time per frame
Recognitions total / today	Counter
Unique identities recognised today	Gauge
Registrations today	Counter

Table 7: Prometheus metrics exposed by the Flask API

Grafana renders preconfigured dashboards on port 3001. SMTP alerting is configured for threshold violations (e.g., error rate $> 5\%$ or data drift detected), satisfying the guideline requirement to configure alerts if error rates exceed 5%.

Node-exporter and DCGM-exporter provide CPU/memory and NVIDIA GPU metrics respectively, giving full-stack observability.

6.2 The Feedback Loop — Ground Truth Capture

Per the guideline requirement to implement a mechanism to log ground truth labels for real-world performance decay:

- The `/api/report_misclassification` endpoint accepts HITL correction reports from the UI.
- Each report is persisted to the `misclassified_faces` SQLite table with the correct label, image crop, and timestamp.
- The misclassification count is exposed as a Prometheus Gauge, feeding the Grafana dashboard.

6.3 Data Drift Detection

The `prepare_data.py` stage computes and saves baseline pixel statistics (mean, variance, distribution) per split to `data/{dataset}_baseline_stats.json`. These baselines serve as the reference distribution for incoming data drift comparison in subsequent training runs.

6.4 Dual-Trigger Re-training Policy

Re-training is triggered by two independent conditions, as required by the guidelines:

1. **Misclassification threshold breach**: When the HITL count exceeds `MISCLASSIFY_THRESHOLD` (default: 2), the Flask API sends a JWT-authenticated POST to Airflow's `/api/v2/dags/{dag_id}/c` endpoint, triggering the `face_recognition_pipeline` DAG immediately. The trigger payload includes `TRIGGERED_BY=misclassification_threshold`, which reduces the Optuna sweep to 1 trial for fast fine-tuning.
2. **Weekly schedule**: The DAG runs on a weekly cadence regardless of threshold, provided at least one misclassification has been logged. This keeps the model current with gradual appearance changes in low-traffic deployments.

6.5 Post Re-training Lifecycle

After each successful re-training and model promotion:

- **Misclassification purge**: All records in the `misclassified_faces` table are deleted, resetting the threshold counter and preventing stale errors from biasing future re-training.
- **Embedding re-computation**: A full re-embedding pass over all saved registration images is performed per enrolled person, correcting for the embedding shift introduced by the new model weights.

7 Architecture Overview — The Three-Plane Model

7.1 Design Philosophy

Three overarching principles guided every architectural decision:

1. **Separation of Concerns**: The UI, inference engine, training pipeline, and observability stack communicate exclusively through well-defined interfaces (REST, MLflow registry, DAG triggers). Each plane can be modified, scaled, or replaced without cascading changes.

2. **Operational Autonomy**: The system detects model degradation via HITL misclassification feedback and triggers its own re-training cycle without manual intervention.
3. **Reproducibility & Auditability**: All hyperparameters, metrics, and artefacts are tracked in MLflow. A DVC pipeline captures data lineage. Every training run can be fully reproduced from `params.yaml` and the registered dataset revision.

7.2 The Three Planes

Plane	Core Services & Role	Communication Interface
Inference Plane	Flask API (`flask-api:2000/8080`) — real-time face detection, embedding extraction, cosine matching, HITL capture	REST/MJPEG inbound; MLflow read; Airflow trigger outbound
Training Plane	Airflow DAG + Docker Trainer + Optuna — scheduled & event-driven training, HPO, model registration	DockerOperator; MLflow write; Flask `/reload` notify
Registry Layer	MLflow Server (port 5000) — experiment logging, Staging → Production promotion, artefact storage	HTTP from Trainer (write) and Flask API (read)
Observability Plane	Prometheus + Grafana + node/dcgm exporters — system & GPU metrics, latency histograms, SMTP alerting	Scrape `/metrics`; dashboards on :3001

Table 8: The three functional planes and their communication contracts

7.3 Why Separate Inference from Training?

Separating inference from training is the single most consequential architectural decision. In a monolithic design, re-training would require taking the inference service offline. By decoupling the two via the MLflow Model Registry, the inference plane can continue serving requests while the training plane runs an Optuna sweep in a separate Docker container. When the new model is promoted to Production, the Flask API hot-reloads it with zero downtime.

8 Programming Paradigm

8.1 Object-Oriented Components

Core ML components are class-based, following PyTorch idioms:

- `LFWIdentityDataset`, `LFWPairDataset` extend `torch.utils.data.Dataset` (`dataloader.py`).

- `PKSampler` extends `torch.utils.data.Sampler`.
- `SiameseEncoder`, `SiameseNetwork` extend `torch.nn.Module` (`model.py`).
- `TripletLoss` extends `torch.nn.Module` (`utils.py`).
- Flask application uses thread-safety class instances (`_model_lock`, `threading.Event`).

8.2 Functional / Procedural Components

Pure functions handle stateless transformations:

- Data: `get_transform()`, `scan_lfw()`, `split_identities()`, `merge_misclassified_into_train()`
- Config: `resolve_params()`, `apply_params()`.
- Inference: `get_embedding()`, `db_recognize()`.
- Airflow DAG tasks are Python callables composed via PythonOperator.

8.3 Design Rationale

PyTorch strongly favours class-based Dataset/Model definitions — the framework's training loop expects objects with `__len__`, `__getitem__`, and `forward()` methods. Flask routes are naturally functional: they receive a request, perform a transformation, and return a response with no retained state. The hybrid paradigm achieves high cohesion within each module while remaining immediately navigable for developers familiar with either PyTorch or Flask conventions.

9 Loose Coupling: Frontend vs Backend

A core design requirement is that the frontend UI and backend inference engine must be independent software blocks connected only via REST API calls.

Criterion	Evidence
Separation of concerns	`index.html` communicates via REST only (`fetch()` calls). No shared state or direct function calls between UI and inference engine.
Configurable API base	All env vars (`MLFLOW_TRACKING_URI`, `AIRFLOW_BASE_URL`, `THRESHOLD`) injected via `docker-compose.yaml` — no code changes needed to reconfigure.
Frontend independence	`index.html` can be served by any static host. The Flask backend at :8080 is the sole dependency. CORS can be trivially enabled.
Backend independence	The inference engine operates independently of any UI. `/metrics` is consumed by Prometheus; `/video_feed` by any MJPEG client.
REST-only coupling	All frontend actions (register, delete, flag_retrain) and backend state machine communication is mediated by HTTP POST/GET with JSON payloads.

Table 9: Loose coupling compliance matrix

10 MLOps Guideline Compliance Review

This section explicitly maps every guideline requirement to its implementation.

10.1 Core Principles

Principle	Implementation
Automation	Airflow DAG automates all stages: data prep → training → registration → evaluation → reload. Weekly schedule + event trigger.
Reproducibility	`params.yaml` + MLflow `run_id` pair fully reproduces any run. DVC tracks data lineage.
Continuous Integration	DVC pipeline (`MLproject`) enables `dvc repro` for CI. GitHub Actions / GitLab CI can consume this.
Monitoring & Logging	Prometheus + Grafana stack. Structured logging via `logging` module in all source files.
Version Control	`params.yaml` committed to Git. MLflow tracks all model versions. DVC tracks dataset versions.
Environment Parity	Dockerfile.train, Dockerfile.api, Dockerfile.airflow ensure identical environments.

Table 10: Core MLOps principle compliance

10.2 Data Lifecycle

Requirement	Implementation
Automated data validation	`prepare_data.py`: schema check, min-images constraint, corruption sampling.
Version-control data scripts	`prepare_data.py`, `dataloader.py` in Git; DVC stages in `dvc.yaml`.
Drift baselines	`compute_baseline_stats()` writes pixel mean/std/variance to `baseline_stats.json`.
Data manifest	`write_manifest()` produces `manifest.json` with exact split counts.
DVC metrics	`data_metrics.json` written for `dvc metrics show`.

Table 11: Data lifecycle compliance

10.3 Model Development

Requirement	Implementation
Experiment tracking	MLflow logs params, metrics, and model artefacts for every run.
Model versioning	MLflow Model Registry: Staging → Production promotion.
Containerised training	`Dockerfile.train` + Docker Compose `trainer` service.
Resource optimisation (no cloud)	Backbone freezing, gradient clipping, CUDA memory management.
Hyperparameter tracking	Optuna sweeps, all params logged to MLflow.

Table 12: Model development compliance

10.4 Deployment

Requirement	Implementation
Multi-container Docker Compose	8 services: flask-api, trainer, mlflow, airflow×3, prometheus, grafana.
Health check endpoints	`/health` on flask-api; Airflow API health; Docker `healthcheck` directives.
Model serving REST API	16 endpoints in `app.py` including `/video_feed`, `/api/register`, `/metrics`.
Rollback mechanism	MLflow staging gate: model only promoted if metrics improve on Production.
CI/CD pipeline	DVC pipeline (`MLproject`) + Airflow DAG provide end-to-end automation.

Table 13: Deployment compliance

10.5 Monitoring

Requirement	Implementation
Performance monitoring	Prometheus scrapes latency histograms, recognition counters.
Ground truth feedback loop	`/api/report_misclassification` captures HITL corrections with labels.
Data drift detection	Baseline stats computed per split; misclassification crops feed next training cycle.
Automated re-training	Dual trigger: HITL threshold breach + weekly Airflow schedule.
Alerting (error rate ¿ 5%)	Grafana SMTP alerts configured for threshold violations.
Model version management	MLflow Registry Staging/Production stages; automatic promotion logic.

Table 14: Monitoring compliance

11 Technology Stack

Category	Technology
Deep Learning	PyTorch 2.x, torchvision (ResNet50 backbone)
Data Engineering	Apache Airflow 3.1.8, DVC
Experiment Tracking	MLflow 2.11.1
Hyperparameter Optimisation	Optuna (TPE sampler)
Inference API	Flask 2.x
Face Detection	MediaPipe
Containerisation	Docker, Docker Compose
Monitoring	Prometheus, Grafana, node-exporter, dcgm-exporter
Databases	PostgreSQL 13 (Airflow), SQLite (faces + misclassifications)
Version Control	Git, DVC, MLflow Model Registry
GPU Telemetry	NVIDIA DCGM Exporter

Table 15: Full technology stack

12 Key Operational Behaviours

12.1 Face Registration and Image Persistence

During registration, MediaPipe detects the face bounding box, the crop is passed through ResNet50 to produce an L2-normalised embedding, and the embedding is stored in SQLite. The raw captured image is also saved to disk under the person's directory — ensuring each enrolled identity's real-world appearance contributes to future fine-tuning rounds via `merge_misclassified_into_train()`.

12.2 Local Live Inference

All inference runs within the `flask-api` container. No frames, crops, or embeddings are transmitted to external servers. The pipeline is: MediaPipe face detection → ResNet50 embedding extraction → cosine similarity search over SQLite embedding store. This prioritises low latency and ensures biometric data never leaves the deployment environment.

12.3 Post Re-training: Misclassification Cleanup

After a successful re-training and model promotion, all misclassification records are deleted from the SQLite audit table. This resets the threshold counter, prevents stale errors from biasing future training, and avoids positive-feedback loops.

12.4 Post Re-training: Embedding Re-computation

Following model promotion, the system performs a full re-embedding pass over all saved registration images for every enrolled person. This corrects the embedding shift introduced by the updated weights. Re-computation is performed per-person with independent error handling so that one corrupt directory does not abort the re-embedding of others.

12.5 Parameter Resolution Priority

Configuration follows a two-tier priority chain (highest to lowest):

`params.yaml` > `config.py defaults`

Implemented in `resolve_params()` / `apply_params()` in `main.py`.

13 Configuration Management

`params.yaml` is the **single source of truth** for all pipeline hyperparameters. It is committed to Git, making any change detectable by DVC for cache invalidation.

```yaml
prepare:
  train_ratio: 0.70
  val_ratio:   0.15
  seed:        42
  min_images_per_identity: 2
  dataset_name: lfw

train:
  n_trials:           1
  n_epochs_per_trial: 20
  lr_min:  1e-5
  lr_max:  1e-5
  margin_min: 0.5
  margin_max: 0.5
  mining_choices: "semi"
  batch_size:  32
  embedding_dim: 128
  dropout_rate:  0.3
```

```
  warmup_epochs: 6

inference:
  threshold:                  0.8
  misclassify_threshold:      2
  model_reload_interval_secs: 60
```

Listing 1: params.yaml — hyperparameter configuration

14 Security Considerations

- **Data at rest**: Face embeddings and misclassification crops are stored within Docker volumes, scoped to the deployment environment.
- **Data in transit**: All Flask API communication is local within the Docker bridge network (`mlops-net`).
- **Airflow authentication**: JWT-authenticated DAG triggers from the Flask API (`AIRFLOW_USERNAME` / `AIRFLOW_PASSWORD` via env vars).
- **Biometric data locality**: No face crops, embeddings, or video frames are transmitted to external servers — a fundamental privacy guarantee.
- **Secrets management**: Credentials are injected via environment variables in `docker-compose.yaml` rather than hardcoded in source.

15 Testing Strategy

Per the guideline requirement to implement unit, integration, and end-to-end tests:

- **Unit tests**: `TestSiameseEncoder.test_backbone_frozen` verifies that backbone parameters have `requires_grad=False` during Phase 1.
- **Data pipeline tests**: Validation that `split_identities()` produces disjoint identity sets with correct ratios.
- **Integration tests**: The Airflow DAG task dependencies enforce integration-level ordering (`prepare_data → check_should_train → train_model → register_model → evaluate_and_promote → notify_flask`).
- **Health check monitoring**: Docker Compose `healthcheck` directives provide ongoing liveness/readiness checks in production.

16 Continuous Improvement

The system's self-improvement cycle follows this loop:

1. **Serve**: Flask API performs real-time inference on live video.
2. **Observe**: Prometheus captures latency, misclassification rate, recognition counts.
3. **Feedback**: Users report misclassifications via the UI (HITL).

4. **Trigger**: Threshold breach or weekly schedule triggers Airflow DAG.
5. **Retrain**: Optuna sweep finds optimal hyperparameters for the updated data distribution.
6. **Evaluate**: New model is compared against Production baseline.
7. **Promote**: If improved, model is advanced to Production; Flask API hot-reloads.
8. **Purge**: Misclassification records are cleared; embeddings are recomputed.
9. Back to Step 1.

Per the guidelines, this loop is sustained by staying current with MLOps advancements and fostering a culture of experimentation through the Optuna sweep mechanism.

17 Conclusion

The MLOps Face Recognition System demonstrates a mature, production-oriented architecture with comprehensive adherence to the MLOps guidelines. Key achievements:

- **Full lifecycle automation**: Every stage from data preparation to deployment is automated via Airflow DAGs and DVC pipelines.
- **Zero-downtime upgrades**: Model hot-reload via MLflow registry and Flask re-embedding pass.
- **Privacy-by-design**: All biometric processing is local; no cloud dependencies.
- **Self-improving**: Dual-trigger re-training policy responds to both sudden degradation and gradual drift.
- **Full observability**: Prometheus + Grafana + SMTP alerting for all system planes.
- **Reproducibility**: `params.yaml` + MLflow `run_id` pair reproduces any historical training run.

The three-plane architecture (Inference, Training, Registry+Observability) connected exclusively via REST, MLflow registry, and Airflow DAG triggers satisfies all loose-coupling and separation-of-concerns criteria. The ResNet50 backbone fine-tuned with triplet loss on LFW, augmented by real-world HITL crops, provides a strong and continuously improving embedding foundation suitable for production deployment on commodity hardware.